

Sisteme de operare

Laborator 14

Semestrul I 2025-2026

Laborator 14

- programare client-server TCP/IP
 - demoni
 - gestionarea mesajelor de eroare ale demonilor
 - server HTTP elementar

Demoni

- procese care ruleaza in background si nu sunt asociate cu un terminal de control
- in general porniti de scripturile sistemului de operare la sfarsitul operatiei de boot
- ce se intampla insa cand un server e pornit de utilizator dintr-un shell (terminal)?
 - trebuie sa se dezasocieze de terminal: (1) pt a evita interactiuni nedorite cu controlul proceselor executat de shell, (2) gestiunea sesiunii de terminal sau (3) pt a nu tipari mesajele server pe terminal (avand in vedere ca ruleaza in background)
- exemple demoni
 - serviciile sistem din */etc/rc* pornite cu privilegii de superuser (*inetd/xinetd*, *Web*, *sendmail/postfix*, *syslogd* etc)
 - job-uri *cron/at* invocate de demonul de *cron*
 - servere/programe pornite din terminal

Output pt demoni

- demonii nu au terminal de control => necesita metode specifice pt a afisa mesajele la nevoie (ex mesaje de eroare, urgenta, logging, etc)
- metoda standard foloseste functia *syslog* care trimite mesaje catre demonul *syslogd* (*rsyslogd* pe sisteme Linux)
 - fisier de configurare *syslog.conf* specifica ce trebuie facut cu fiecare mesaj primit de demon:
 - se adauga la sfarsitul unui fisier
 - se logheaza pe consola (*/dev/console*)
 - se forwardeaza catre demonul *syslogd* al altei masini
 - serviciul functioneaza pe portul 514 UDP (v. */etc/services*)
 - la primirea semnalului SIGHUP se reciteste fisierul de configurare (sistemele moderne folosesc comanda *service reload*)
 - ex: `$ kill -HUP <pid syslogd>`
- mesajele se pot trimite direct pe portul 514 UDP, dar uzual se foloseste functia *syslog*

Functia syslog

- in lipsa terminalului de control demonul nu poate folosi functii de genul *fprintf* la *stderr*
- logarea mesajelor se face cu functia *syslog*

```
#include <syslog.h>
```

```
void syslog(int priority, const char *message, ...);
```

- primul argument este o combinatie intre *nivel* si *facilitate* (*level/facility*)
- mesajul (al doilea parametru) este un string de formatare ca la *printf*
 - *%m* poate fi folosit pt a loga mesajul de eroare corespunzator valorii curente a *errno*
- mesajele logate au un nivel intre 0 si 7
- daca utilizatorul nu specifica nici un nivel se foloseste implicit LOG_NOTICE (nivel 5)
- toate mesajele de un anumit nivel se trateaza la fel

Functia syslog (cont.)

- mesajele logate contin o *facilitate* care identifica procesul care trimite mesajul
- valoarea implicita pentru *facilitate* este LOG_USER
- ex: mesaj de eroare demon care nu reuseste sa redenumiasca un fisier

syslog(LOG_INFO | LOG_LOCAL2, "rename (%s,%s): %m", file1, file2);

- valoarea facilitatii permite ca toate mesajele venite de la o anumita facilitate sa fie tratate la fel in *syslog.conf*
 - ex:

<i>kern.*</i>	<i>/dev/console</i>
<i>local7.debug</i>	<i>/var/log/cisco.log</i>

Functia syslog (cont.)

- primul apel *syslog* creeaza un socket Unix UDP care se conecteaza la portul 514 si care ramane conectat pana cand se termina procesul
- alternativ, se pot folosi functiile *openlog/closelog*

```
#include <syslog.h>
```

```
void openlog(const char *ident, int options, int facility);
```

```
void closelog((void);
```

- *ident* este un string care va prefixa toate mesajele logate prin *syslog*
- *options*: OR logic intre mai multe constante (v. man page)
- daca se specifica *facilitatea* la *openlog*, ulterior cand se apeleaza *syslog* nu se mai paseaza ca prim parametru decat *nivelul*
- mesajele se pot loga si cu comanda *logger* (din linie de comanda)

Structura demoni

- (1) detasarea de terminal
 - se apeleaza *fork*, procesul parinte se termina imediat si copilul continua
=> shell-ul crede ca s-a terminat comanda, ca atare copilul ruleaza automat in background
 - copilul mosteneste GID-ul parinte dar are propriul PID, fapt ce face ca el sa nu poata fi liderul grupului de procese
 - copilul creeaza o noua sesiune si devine lider de grup in noua sesiune (apel *setsid*)
 - copilul nu mai are terminal de control in acest moment
 - copilul se protejeaza de semnalul SIGHUP si apeleaza *fork* din nou
 - primul copil, acum parinte, termina si copilul sau (al doilea copil) continua sa ruleze
 - al doilea apel *fork* garanteaza ca demonul nu poate capata un terminal de control daca decide in viitor sa deschida un terminal
 - cand un lider de sesiune fara terminal deschide un terminal, acel terminal devine terminalul de control al liderului sesiunii
 - al doilea apel *fork* garanteaza ca al doilea copil nu mai e lider de sesiune si nu mai poate obtine un terminal de control
 - SIGHUP trebuie ignorat pt ca atunci cand liderul de sesiune (primul copil) se termina, toate procesele din sesiune (al doilea copil) primesc semnalul SIGHUP

Structura demoni (cont.)

- (2) schimbarea directorului de lucru curent
 - demonul schimba directorul de lucru curent in / (sau alt director la alegere)
 - motive: (1) fisierele core se salveaza in directorul de lucru curent
 - (2) daca demonul continua sa lucreze in directorul curent, sistemul de fisiere respectiv nu va putea fi dezinstale cu *umount*
 - implicatii majore pt sisteme de fisiere distribuite, ex: NFS
- (3) inchiderea tuturor descriptorilor de fisiere deschisi
 - se inchid toti descriptorii de fisiere mosteniti de la procesul care a creat demonul (in mod normal, shell-ul)
- (4) redirectarea stdin, stdout si stderr la */dev/null*
 - garanteaza ca descriptorii de fisiere corespunzatori sunt deschisi, dar citirea intoarce EOF iar scrierea se pierde
 - necesar pt ca functiile de biblioteca care presupun folosirea acestor descriptori sa nu esueze
- (5) utilizarea *syslogd* pentru logarea erorilor
 - apel *openlog*, in mod normal cu numele procesului (*argv[0]*) ca parametru

Sisteme de operare

Laborator 14

Demoni si servere Web

1. Creati un nou subdirector *lab14/* in structura de directoare a laboratorului creata anterior (*SO/laborator*) si subdirectoarele aferente *doc src* si *bin*. Nu uitati sa actualizati variabila de mediu *PATH* pentru a include directorul *SO/laborator/lab14/bin*.

2. Scrieti o functie *daemon_init* care transforma un server intr-un demon. Cititi cu atentie slide-urile de laborator si implementati fiecare din pasii necesari. Semnatura functiei este urmatoarea:

```
int daemon_init(const char *pname, int facility);
```

Functia intoarce valoarea -1 daca esueaza (i.e., nu poate transforma serverul in demon) sau 0 in caz de succes. Primul parametru este numele serverului iar al doilea este valoarea de facilitate folosita de *syslog*.

Pentru a testa functia scrieti un mic program **daemon.c** care apeleaza *daemon_init*, scrie PID-ul demonului (al doilea copil) intr-un fisier **daemon.pid** pe care il creeaza intr-un director in care utilizatorii au drept de scriere (eg, */tmp*), afiseaza cu *syslog* un mesaj "*Daemon started ...*" si apoi intra intr-o bucla infinita (eventual apeland *sleep* la fiecare iteratie). Verificati cu comanda *ps* ca demonul pe care l-ati creat nu are asociat nici un terminal. Terminati executia demonului cu comanda *kill*, de exemplu ca mai jos:

```
$ kill `cat /tmp/daemon.pid`
```

Obs: in mod normal la terminarea demonului ar trebui folosit *syslog* pentru a scrie un mesaj de tip "*Daemon stopped ...*" si ar trebui inchis log-ul. De asemenea ar trebui sters si fisierul **daemon.pid**.

Indicatie: pentru a vedea mesajele scrise cu *syslog* puteti deschide un terminal separat in care sa rulati comanda de mai jos:

```
$ tail -f /var/log/syslog
```

3. Scrieti un program **http-echod.c** care modifica serverul **echod-tcp.d** scris in laboratorul anterior pentru a servi cereri TCP de la clienti pe portul 8080.

http-echod nu face decat sa trimita inapoi ca raspuns clientului propria lui cerere, dar in acest scop serverul trebuie sa formeze un raspuns corect HTTP, care presupune existenta unui header. Raspunsul serverului este un text care trebuie sa includa un header de tipul urmatoar:

```
"HTTP/1.1 200 OK\n"
"Content-Type: text/html; charset=utf-8\n"
```

Header-ul este despartit de restul raspunsului serverului HTTP printr-un dublu caracter *newline*.

http-echod copiaza cererea primita de la client, trimite header-ul de mai sus catre client urmat de copia cererii clientului si inchide conexiunea. In acest scop, **http-echod** foloseste apelul sistem *writew* pentru a trimite header-ul si cererea clientului.

Folositi drept client de Web orice browser (cel mai bine e sa testati cu mai multe browsere). Ca sa accesati serviciul tipariti in browser URL-ul urmator:

http://localhost:8080

Ce anume cititi in browser? Cum arata o cerere client de HTTP? Cautati informatii pe net despre semnificatia campurilor pe care le identificati in cererile clientilor.

Odata ce aveti un server functional, transformati-l in demon folosind functia *daemon_init* scrisa la exercitiul anterior.

Obs: Dupa ce serverul se transforma in demon, **nu se pot folosi printf/scanf & co. pt a scrie/citi mesaje pe ecran/de la tastatura!** Nu se poate folosi decat functia *syslog* iar mesajele vor aparea in */var/log/syslog*.

De asemenea, avand in vedere ca demonul e detasat de terminal, e util sa salvati PID-ul sau intr-un fisier **http-echod.pid** la fel ca la exercitiul anterior si pe care sa-l folositi pentru a termina serverul la nevoie (in general, pentru a-i trimite semnale) ca mai jos:

```
$ kill `cat /tmp/http-echod.pid`
```

4. Scrieti un program **httpd.c** care extinde **http-echod** a.i. dupa ce citeste cererea clientului sa identifice in cererea acestuia documentul Web solicitat atunci cand clientul a accesat URL-ul *http://localhost:8080*.

Serverul trebuie sa formeze un raspuns corect HTTP pe care browserul sa-l inteleaga. In acest scop puteti folosi functia *http_response* pe care ati scris-o in laboratorul 11:

```
int http_response(char *filename, int fd);
```

De data aceasta veti apela functia cu descriptorul de socket de serviciu ca parametru. O alta posibilitate este sa folositi superserverul Internet *inetd* pe care l-ati scris in laboratorul 13 si atunci **httpd.c** va fi un program server care scrie la *stdout*, caz in care veti apela functia *http_response* cu file descriptorul 1 asa cum ati procedat in laboratorul 11.

Observatie: daca intr-un URL nu apare o cerere explicita pentru un document web anume (i.e., un anumit nume de fisier html) ca in URL-ul de mai sus, presupunerea implicita a serverului este ca se solicita un document numit *index.html* (sau *index.php* daca vorbim de documente dinamice).

Indicatie: pentru simplitate, la inceput folositi un fisier *index.html* care redirecteaza clientul catre adresa <https://fmi.unibuc.ro>, ca mai jos:

```
<head>
    <meta http-equiv="Refresh" content="0; URL=https://fmi.unibuc.ro/" />
</head>
```

5. Instruiti browserul client sa foloseasca un URL care refera un document HTML ceva mai sofisticat, de pilda fisierul *bosch.html* de mai jos:

```
<html>
<head>
<title>Example</title>
```

```

</head>
<body>
<h1>Ship of fools</h1>

</body>
</html>

```

Cum se schimba interactiunea clientului cu serverul **httpd.c** atunci cand tipariti in browser URL-ul *http://localhost:8080/bosch.html*? De ce nu functioneaza serverul pe care l-ati scris la punctul anterior? Ce modificari trebuie sa faceti pentru ca **httpd.c** sa poata servi asemenea cereri?

6. Extindeti programul **httpd.c** anterior pentru a servi cereri HTTP pentru documente dinamice pe portul 8080. Termenul de document dinamic se refera punctual, in cazul acestui exercitiu, la scripturi *php* care sunt solicitate de cereri client care folosesc URL-uri de tipul *http://localhost:8080/my-php-script*, unde *my-php-script.php* este un script *php*.

Indicatii: E posibil sa fie nevoie sa instalati *php* pe calculatorul pe care lucrat. Pentru testare puteti folosi de exemplu un script *my-php-script.php* de genul urmatoar:

```

<!DOCTYPE html>
<html>
<head>
<title>Example</title>
</head>
<body>
<?php
    echo "Hi, I'm a PHP script!";
?>
</body>
</html>

```

sau puteti scrie propriul script *php* care genereaza cod html. Inainte de a testa in retea scriptul, testati-l in terminal folosind comanda *php*. Pentru un test ceva mai complex puteti transforma documentul static folosit in exercitiul 5 de mai sus intr-un script *php* de felul urmatoar:

```

<?php
    echo "<!DOCTYPE html>
<html>
<head>
<title>Example</title>
</head>
<body>
<h1>Ship of fools</h1>

</body>
</html>";
?>

```

7. Extindeti serverul **httpd.c** in continuare pentru a trata cereri HTTP POST pe portul 8080. In acest scop, modificati codul **httpd.c** a.i. sa poata servi cereri client care folosesc URL-uri de tipul *http://localhost:8080/adding*, unde *adding.php* este un script *php* care creeaza un form cu doua

campuri si un buton de submit intitulat “Add” care atunci cand este apasat de utilizator aduna cele doua numere introduse de utilizator in cele doua campuri si afiseaza suma lor.

Indicatie: Pentru a afla formatul unei cereri POST, logati cererile POST primite de server in */var/log/syslog* cu ajutorul functiei *syslog*.

Odata ce ati inteles formatul unei cereri POST trebuie sa parsati cererea POST pentru a afla valorile parametrilor scriptului *adding.php* introduse de utilizator.

Indicatie: puteti folosi ca exemplu urmatoarea implementare a scriptului *adding.php*:

```
<html>
  <body>
    <form method="post">
      Enter First Number:
      <input type="number" name="number1" /><br><br>
      Enter Second Number:
      <input type="number" name="number2" /><br><br>
      <input type="submit" name="submit" value="Add">
    </form>
  </body>
</html>

<?php
    if(isset($_POST['submit']))
    {
        $number1 = $_POST['number1'];
        $number2 = $_POST['number2'];
        $sum = $number1+$number2;
        echo "The sum of $number1 and $number2 is: ".$sum;
    }
    ?>
```